

第4章 函数与方法

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Go 的函数定义语法与多返回值,理解 `(T, error)` 是 Go 的错误处理灵魂
- 掌握命名返回值与裸 `return` 的语义
- 透彻理解"全部值传递":`slice/map` 传的是头结构副本,函数内修改元素会影响外部,修改长度不会
- 掌握函数值、匿名函数与闭包:捕获语义、何时该传参数、与 C 函数指针的对照
- 掌握 `defer` 的执行时机(LIFO)、参数立即求值,用 `defer` 管理资源
- 掌握可变参数与 `...` 展开
- 理解 `goroutine` 栈自动增长带来的"递归安全",以及 Go 没有默认参数/重载的事实

💡 从 C 到 Go:本章主题如何迁移

C 的函数是"一段代码 + 一堆参数 + 一个返回值"。想返回多个结果?要么用输出参数(指针),要么传一个结构体指针让函数填。想传函数?用函数指针,但函数指针不能捕获上下文,回调里要带 `void* ctx` 手工搬运环境。错误处理? `errno` 全局变量,检查起来全靠自觉。

Go 对这四件事各给了一个干净的答案。**多返回值**让 `result, err := f()` 成为日常;错误不再是全局状态而是普通值。**函数值**是一等公民,可以赋给变量、放进 `map`、传给别的函数;配合**闭包**,函数可以"记住"创建时的环境——这是函数指针+ `void* ctx` 的优雅替代。**defer** 让"函数退出时必做清理"成为语法级保证,比 C 的手工 `goto cleanup` 更可靠。而**递归**:C 的递归栈固定(通常几 MB),深递归秒崩;Go 的 `goroutine` 栈 2KB 起步、自动增长,深递归安全得多。

还有两个"没有":Go **没有默认参数**(用可变参数或构造选项结构体替代)、**没有函数重载**(同名不同参),却也不需要 C 的 `_Generic` 绕——泛型(第 10 章)和不同函数名把问题解决得更干净。

一句话主线: C 的函数是"工具",Go 的函数是"公民"——能传递、能嵌套、能记住环境,错误处理融入类型系统而非游离在外。

📦 前置知识自查

- C 的函数声明/定义/调用、形参实参、`return` 语句、`main` 函数
- C 的传值语义与"用指针模拟引用"的参数传递
- C 的函数指针、回调函数、`qsort` 式的 `void*` 上下文
- C 的 `errno` 错误处理模式及其缺陷
- 递归的基本概念(阶乘/斐波那契)

本章学习路线

1. **第一阶段**:§4.1–§4.3,把"值传递"的四种情况(基础类型/数组/切片/map/指针)全部跑一遍
2. **第二阶段**:§4.4–§4.6,重点练闭包捕获与 defer 的 LIFO、立即求值
3. **第三阶段(实验)**:§4.7–§4.8 与章末实验:可配置的统计管线,用函数值与闭包搭建 filter/map/reduce
4. **验收标准**:能不看资料回答"slice 传进函数改长度为什么无效""defer 什么时候求值参数"

本章知识导图

- §4.1 函数定义与调用:func 语法、参数、多返回值
- §4.2 参数传递:全部值传递(基础类型/数组/切片/map/指针对照)
- §4.3 命名返回值与裸 return
- §4.4 函数值、匿名函数与闭包(与 C 函数指针对照)
- §4.5 defer:延迟执行、LIFO、参数立即求值
- §4.6 可变参数与 ... 展开
- §4.7 递归与栈自动增长
- §4.8 错误返回约定 (T, error) 与 main 组织

§4.1 函数定义与调用:多返回值

Go 的函数语法: `func 名字(参数)(返回值) { }`。类型写在名字后面(与变量一致);多个同类型参数可合并: `func add(a, b int)`。最重要的特性是**多返回值**:

示例 4-1 多返回值

```

package main

import "fmt"

// 返回商与余数
func divmod(a, b int) (int, int) {
    return a / b, a % b
}

// 返回计算结果与错误:Go 的黄金惯例
func sqrt(x float64) (float64, error) {
    if x < 0 {
        return 0, fmt.Errorf("负数没有平方根: %v", x)
    }
    // 牛顿迭代法
    z := 1.0
    for i := 0; i < 10; i++ {
        z -= (z*z - x) / (2 * z)
    }
    return z, nil
}

func main() {
    q, r := divmod(17, 5)
    fmt.Println("商", q, "余", r)

    v, err := sqrt(9)
    if err != nil {
        fmt.Println("错误:", err)
        return
    }
    fmt.Println("sqrt(9) =", v)

    // 只关心部分返回值:用 _ 丢弃
    _, e := sqrt(-1)
    fmt.Println("对 -1:", e)
}

```

★ 重点:(T, error) 是 Go 的错误处理灵魂

C 用 `errno` 全局变量,失败后必须立刻检查且容易被覆盖;Go 把错误作为**普通返回值**,与成功值并列。`v, err := f()` 之后 `if err != nil` 是 Go 代码中出现频率最高的三行。生产铁律:**错误必须被检查**——`go vet` 的 `errcheck` 类工具会标记被丢弃的错误返回值,不要用 `_` 丢掉错误。

§4.2 参数传递:全部值传递

Go 的一切参数都是**按值传递**——与 C 相同。但"值"是什么,取决于类型:基础类型传副本;数组传整个副本(第 1 章);切片传"指针+`len`+`cap`"三个数的副本;`map` 传"指向哈希表的指针"的副本。因此:

C 的写法	Go 的写法
值传递:函数内修改不影响外部	基础类型/数组:同样,修改不影响外部
传指针才能改外部变量	传 <code>*T</code> 才能改外部变量(指针也是值拷贝,拷贝的是地址)
数组退化为指针,函数内能改数组	数组按值传递,函数内改数组不影响外部——要改请传 <code>*[N]T</code> 或切片
无对应(无 GC 切片)	切片:函数内改元素影响外部;append 导致的重分配不影响外部

示例 4-2 值传递的四种情况

```
package main

import "fmt"

// 基础类型:副本,不影响外部
func addOne(n int) {
    n++
}

// 数组:整个拷贝,不影响外部
func zeroArr(a [3]int) {
    a[0] = 0
}

// 切片:改元素影响外部;append 改变长度不影响外部
func mutate(s []int) {
    s[0] = 99          // 修改底层数组,外部可见
    s = append(s, 1, 2) // 只改本函数的副本
    s[1] = -1          // 若 append 触发了重分配,这里的修改外部也看不见
}

// map:改内容影响外部(共享哈希表)
func addKV(m map[string]int) {
    m["新增"] = 1
}

func main() {
    n := 1
    addOne(n)
    fmt.Println("addOne 后 n =", n) // 1

    arr := [3]int{1, 2, 3}
    zeroArr(arr)
    fmt.Println("zeroArr 后 arr =", arr) // [1 2 3]

    s := []int{10, 20, 30}
    mutate(s)
    fmt.Println("mutate 后 s =", s) // [99 20 30]:只看到元素修改

    m := map[string]int{}
    addKV(m)
    fmt.Println("addKV 后 m =", m) // map[新增:1]
}
```

⚠ 易错点 4-1:"函数内 append 了,外面长度没变"

`append` 可能重分配底层数组(容量不足时),此时本函数的切片头指向新数组,外部的切片头仍指向旧数组——所以"函数内 `append` 后长度不变"是正确行为。想让函数"返回增长后的切片",把切片作为返回值传回:`s = mutate(s)`。生产上这个模式无处不在,第 8 章、第 9 章的示例会反复出现。

🚀 工程建议:什么时候传指针?

三类情况:①函数要修改调用方的值(基本类型、结构体);②结构体很大,避免整体拷贝(几百字节以上值得考虑);③语义上"这个对象是共享的"(如连接、缓冲区)。其余情况一律传值——Go 社区默认传值,简单直接,还能顺便让数据不可变(第 14 章深化)。

§4.3 命名返回值与裸 `return`

返回值可以像参数一样**命名**。命名返回值在函数一开始就被初始化为零值,函数体可以直接给它们赋值,最后 `return` 不加表达式(裸 `return`)即可。命名返回值在 `defer` 场景特别有用:`defer` 可以读取/修改最终返回值(§4.5 有实例)。

示例 4-3 命名返回值

```
package main

import (
    "fmt"
    "strings"
)

// 命名返回值:声明了 result 与 err
func splitName(full string) (first string, err error) {
    parts := strings.SplitN(full, " ", 2)
    if len(parts) < 2 {
        err = fmt.Errorf("格式错误: %q", full) // 直接给 err 赋值
        return                                // 裸 return:返回 first 与 err
    }
    first = parts[0]
    return
}

func main() {
    f, err := splitName("张三 李四")
    if err != nil {
        fmt.Println("错误:", err)
        return
    }
    fmt.Println("姓:", f)

    if _, err := splitName("只有一段"); err != nil {
        fmt.Println("预期错误:", err)
    }
}
```

⚠ 易错点 4-2:裸 return 与命名返回值混用要克制

命名返回值 + 裸 return 适合短函数(如上面的判错提前退出);超过 20 行或逻辑分叉多时,裸 return 会让读者找不到"到底返回了啥",此时显式写 `return v, err` 更清晰。这也是 Go 社区 code review 的常见意见。

§4.4 函数值、匿名函数与闭包

Go 的函数是**一等公民**:可以赋给变量、作为参数、作为返回值、存进 map。C 也有函数指针,但函数指针**不能捕获环境**——回调必须靠 `void* ctx` 手工传上下文。Go 的**闭包**直接"记住"了创建时的变量:这本质上就是把 C 的 `ctx` 自动化了。

示例 4-4 函数值:从函数指针到一等公民

```
package main

import "fmt"

func add(a, b int) int { return a + b }
func sub(a, b int) int { return a - b }

// 高阶函数:接收函数作为参数
func apply(a, b int, op func(int, int) int) int {
    return op(a, b)
}

func main() {
    // 函数值赋给变量
    var op func(int, int) int
    op = add
    fmt.Println(op(3, 4)) // 7

    // 函数放进 map:分发表
    ops := map[string]func(int, int) int{
        "+": add,
        "-": sub,
    }
    fmt.Println(ops["+"](10, 5), ops["-"](10, 5))

    // 匿名函数 + 立即调用
    result := func(x int) int { return x * x }(9)
    fmt.Println(result)

    // 高阶函数
    fmt.Println(apply(6, 2, sub))
}
```

示例 4-5 闭包:记住创建时的环境

```

package main

import "fmt"

// 计数器工厂:每次调用返回一个独立的闭包
func makeCounter(start int) func() int {
    n := start // 闭包捕获 n
    return func() int {
        n++ // 修改的是捕获的变量,跨调用保留
        return n
    }
}

func main() {
    c1 := makeCounter(100)
    fmt.Println(c1(), c1(), c1()) // 101 102 103

    c2 := makeCounter(0) // 独立环境
    fmt.Println(c2(), c2()) // 1 2
    fmt.Println(c1()) // 104:c1 不受 c2 影响

    // 捕获 vs 传参:每次调用求值的区别
    for i := 0; i < 3; i++ {
        // 传参数:值在调用时固定
        func(x int) { fmt.Print(x) }(i)
    }
    fmt.Println()
}

```

⚠ 易错点 4-3:闭包捕获的是变量,不是值

闭包捕获的是**变量本身**(引用),不是捕获瞬间的值。循环变量捕获 bug(第 3 章)、Python 晚期绑定坑都是同源问题。需要"捕获某个时刻的值"时:①把值作为参数传给匿名函数并立即调用(上面的写法);②在循环体内 `i := i` 复制;③Go 1.22+ 直接依赖新循环语义。

§4.5 defer:延迟执行与资源管理

`defer f()` 把调用推迟到**当前函数返回前**执行。三个必须牢记的语义:

- **LIFO(后进先出)**:多个 `defer` 逆序执行,像堆栈——后打开的资源先关闭
- **参数立即求值**:`defer fmt.Println(x)` 在 `defer` 语句执行的那一行就拷贝了 `x` 的当前值,函数后面改 `x` 不影响输出
- **总是执行**:即使函数中途 `return`、`panic`,`defer` 也保证执行——这正是"资源释放"的可靠出口,等价于 C++ 的 `RAII` 与 C 的 `goto cleanup`

示例 4-6 `defer` 的执行时机与顺序

```

package main

import "fmt"

func demo() {
    fmt.Println("① 函数体开始")
    defer fmt.Println("④ defer 最后执行(LIFO)")
    defer fmt.Println("③ defer 逆序")
    fmt.Println("② 正常语句")
}

func main() {
    demo()

    // 参数立即求值:x 的值在 defer 处就固定了
    x := 1
    defer fmt.Println("defer 时的 x =", x)
    x = 999
    fmt.Println("修改后 x =", x)
}

```

示例 4-7 defer 配合命名返回值

```

package main

import "fmt"

// 统计函数执行时间(生产常用模式)
func slowWork() (err error) {
    defer func() {
        fmt.Println("slowWork 返回 err =", err) // defer 能读到最终返回值
    }()
    // 模拟业务
    return fmt.Errorf("业务失败")
}

func main() {
    _ = slowWork()
}

```

⚠ 易错点 4-4:在循环里写 defer,资源积压到函数末尾

defer 在函数返回时执行,不是循环迭代结束时。for { f, err := os.Open(...); defer f.Close() } 处理 10 万个文件,会同时打开 10 万个句柄直到函数结束。生产写法:把循环体抽成单独函数(每个调用一个 defer),或手动关闭。

§4.6 可变参数与 ... 展开

与 C 的 ... (va_list 那套手动 va_start / va_arg)不同,Go 的可变参数是一个切片:func sum(nums ...int) 里 nums 就是 []int,直接遍历即可。调用时用 f(s...) 把切片展开。可变参数可以混普通参数(func log(level string, msgs ...string)),且可变参数必须是最后一个。

示例 4-8 可变参数与展开

```
package main

import (
    "fmt"
    "strings"
)

func logAll(level string, msgs ...string) {
    fmt.Printf("[%s] %s\n", level, strings.Join(msgs, " "))
}

func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

func main() {
    logAll("INFO", "服务启动", "端口=8080", "环境=prod")
    logAll("WARN", "磁盘空间不足")

    fmt.Println(sum(1, 2, 3))    // 6
    data := []int{10, 20, 30}
    fmt.Println(sum(data...))    // 展开切片:60
}
```

§4.7 递归与栈自动增长

C 的线程栈大小固定(通常 1-8MB),深递归(如 10 万层)立刻栈溢出崩溃。Go 的 goroutine 栈从 **2KB** 起步,按需自动增长(上限 1GB),所以深递归安全得多——10 万层斐波那契/树的深度遍历在 Go 里通常没问题。代价是每次增长有少量拷贝开销。

示例 4-9 深递归在 Go 中是安全的

```
package main

import "fmt"

// 深度递归:计算 1+2+...+n(展示深递归能力,生产用公式)
func deepSum(n int) int {
    if n == 0 {
        return 0
    }
    return n + deepSum(n-1)
}

func main() {
    // 100 万层递归:C 里直接栈溢出;Go 里栈自动增长,正常完成
    fmt.Println(deepSum(1000000))
}
```

工程建议:递归仍是生产风险项

栈能自动增长不代表无限:无限递归最终仍会 `panic( goroutine stack exceeds 1000000000-byte limit )`。生产要点:树/图遍历优先用显式栈(切片模拟)或迭代;递归深度与输入规模成正比时,先估算最坏深度。Go 的递归和 C 一样没有尾调用优化(TCO),不要指望"尾递归等于循环"。

§4.8 错误返回约定与 main 组织

Go 程序的形状:C 的 `int main(int argc, char** argv)` 在 Go 里是 `func main()` (无参数无返回值),命令行参数通过 `os.Args` 读取(第 7 章)。业务代码几乎全是"函数 + 错误返回值"的组合,没有异常(第 9 章详述 `panic/recover` 的边界)。一个完整的、生产风格的函数骨架:

示例 4-10 生产风格的函数组织

```
package main

import (
    "errors"
    "fmt"
)

// 业务函数:先校验 → 再处理 → 返回 (值, 错误)
func process(name string) (result string, err error) {
    if name == "" {
        return "", errors.New("名字不能为空")
    }
    if len(name) > 10 {
        return "", fmt.Errorf("名字过长: %d 字符", len(name))
    }
    result = fmt.Sprintf("欢迎, %s!", name)
    return
}

// 组合多个可能失败的操作:错误会沿调用链向上传递
func runAll(names ...string) error {
    for _, n := range names {
        if _, err := process(n); err != nil {
            return fmt.Errorf("处理 %q 失败: %w", n, err) // %w 包装错误(第 9 章)
        }
    }
    return nil
}

func main() {
    fmt.Println(runAll("张三", "李四")) // nil:全部成功
    fmt.Println(runAll("张三", ""))    // 错误,带上下文
}
```

本章速查表

主题	要点
函数语法	<code>func 名(参数)(返回值) { }</code> ; 类型后置; <code>func add(a, b int) int</code>
多返回值	<code>func f() (int, error); v, err := f();</code> 丢弃用 <code>_</code>
命名返回值	函数头命名, 零值初始化; 裸 <code>return</code> 返回它们; <code>defer</code> 可读最终值
参数传递	全部值传递; 切片/map 传头结构副本; 改元素可见, 改长度不可见
数组参数	整体拷贝; 要修改传 <code>*[N]T</code> 或切片
函数值	一等公民; 变量/参数/返回值/map 值; <code>var f func(int) int</code>
闭包	捕获变量(引用非值); 计数器/回调/工厂; 注意循环捕获
defer	函数返回前执行; LIFO; 参数立即求值; <code>panic</code> 时也执行
defer 用途	关文件/解锁/统计耗时/恢复 <code>panic</code> ; 循环内慎用
可变参数	<code>func f(args ...T); f(s...)</code> 展开; 只能放最后
递归	栈 2KB 起步自动增长; 深递归安全; 无 TCO; 无限递归仍 <code>panic</code>
没有的	无默认参数(用可变参数/选项结构体); 无重载(用不同函数名/泛型)
错误约定	<code>return 值, err; %w</code> 包装; <code>if err != nil</code> 必须检查

最小必会清单

- 能写出多返回值函数与 `v, err := f()` 的调用 + 错误检查三行式
- 能解释"切片传进函数, 为什么改元素有效、改长度无效", 并写出正确模式 `s = mutate(s)`
- 能写出命名返回值 + 裸 `return` 的短函数, 并说明何时不要用裸 `return`
- 能写出闭包计数器工厂, 并解释闭包捕获的是变量而非值
- 能说出 `defer` 的三条语义(LIFO、立即求值、必然执行) 并举生产实例
- 能写出可变参数函数与 `s...` 展开调用
- 能解释为什么 Go 里深递归比 C 安全, 以及无限递归的后果
- 能说出 Go 没有默认参数与重载, 及其两个替代方案

自测题

Q 1. 写出 `func swap(a, b int)` 与 `func swap(a, b *int)` 的行为差异, 并解释为何 Go 里常用第一种写法实现"换值"。

✅ 参考答案

传值版交换副本, 外部无变化; 指针版修改外部变量。Go 里常用"返回新值"而不是改参数: `func swap(a, b int) (int, int) { return b, a }`, 调用处 `a, b = swap(a, b)` ——配合多重赋值更简洁, 且不改调用方状态。

Q 2. 下面的代码输出什么? `s := []int{1}; f := func(s []int) { s = append(s, 2) }; f(s); fmt.Println(s)`

✅ 参考答案

输出 [1]。闭包参数 `s` 是切片头副本, `append` 修改的是副本(容量 1 不足,触发重分配),外部 `s` 仍指向原底层数组。

Q 3. defer 的执行顺序:写出下列输出。 `defer fmt.Println("A"); defer fmt.Println("B"); fmt.Println("C")`

✅ 参考答案

先输出 C(正常执行),函数返回前按 LIFO 输出 B、A,最终顺序:C B A。

Q 4. 判断:defer 语句执行时,其参数是"立即求值"还是"返回前求值"?给出一个证明示例。

✅ 参考答案

立即求值。示例: `x := 1; defer fmt.Println(x); x = 999` 输出 1 而不是 999——defer 记录的是语句执行瞬间的参数值。想要"返回时看最新值",要包一层闭包: `defer func() { fmt.Println(x) }()` (闭包捕获变量引用)。

Q 5. 可变参数:如何用 ... 实现"把切片传给可变参数函数"与"把可变参数拼接为切片"?

✅ 参考答案

`sum(s...)` 展开切片为实参;函数内部 `nums` 本身就是切片,可直接 `append(other, nums...)` 拼接。注意不能对非切片(如数组)用 ... 展开。

Q 6. Go 没有函数重载,处理"同名不同参"需求时有哪些替代?(至少两种)

✅ 参考答案

①不同函数名: `parseInt / parseFloat`;②可变参数 `func f(v ...int)`;③泛型(第 10 章): `func Min[T int | float64](a, b T) T`;④选项结构体:参数太多时定义一个 `Options` 结构体,兼顾可读与扩展。

章末实验题:可配置的统计管线

实验 4:函数式风格的统计管线

实验目标:用函数值 + 闭包实现一个可配置的数据处理管线:过滤 → 映射 → 归约,覆盖本章全部知识点。

实验要求:

- 任务 1:实现 `filterFloat(list, keep func(float64) bool)` 高阶函数,遍历用 `range`,返回新切片(覆盖:\$4.4 函数值、\$4.2 切片传参、第 3 章 `range`)
- 任务 2:实现 `mapFloat(list, f func(float64) float64)`,用 `make` 预分配容量(覆盖:\$4.4、第 1 章 `make`)
- 任务 3:实现 `reduceFloat(list, init, f func(float64, float64) float64)` 与字符串版 `reduceStr` (覆盖:\$4.4 高阶函数、\$4.1 多返回值不用但参数类型多样)
- 任务 4:用闭包实现"统计型映射":记录被调用次数;用闭包实现阈值可配置的过滤(覆盖:\$4.4 闭包捕获)
- 任务 5:管线组合:过滤发热体温 → 求平均值;字符串切片 → 拼接(覆盖:\$4.6 拼接、组合)
- 任务 6:主函数用 `v, err := ...` 风格组织,至少一个函数返回 (值, error),并在 `main` 中检查(覆盖:\$4.8 错误约定)

实验环境:Go 1.21+, `go run main.go` 运行;用 `go vet` 检查。

第一步 实现三个纯高阶函数,保持"不修改入参、返回新切片"的纯函数风格

第二步 匿名函数 + 闭包完成过滤条件(阈值变量在闭包内捕获)

第三步 统计调用次数:闭包内捕获的计数变量跨调用保留

第四步 `duce` 的初值语义想清楚:求和用 0.0,求最大值用负无穷或首元素

第五步 后把错误约定加进去:如"空输入返回错误"

✔ 参考答案要点

```

// 函数式风格的统计管线:filter / map / reduce + 闭包
package main

import (
    "errors"
    "fmt"
    "math"
    "strings"
)

// filterFloat 过滤:保留 keep 返回 true 的元素
func filterFloat(list []float64, keep func(float64) bool) []float64 {
    out := []float64{}
    for _, v := range list {
        if keep(v) {
            out = append(out, v)
        }
    }
    return out
}

// mapFloat 映射:逐元素变换
func mapFloat(list []float64, f func(float64) float64) []float64 {
    out := make([]float64, len(list))
    for i, v := range list {
        out[i] = f(v)
    }
    return out
}

// reduceFloat 归约:累积
func reduceFloat(list []float64, init float64, f func(float64, float64) float64) float64 {
    acc := init
    for _, v := range list {
        acc = f(acc, v)
    }
    return acc
}

// reduceStr 字符串版归约
func reduceStr(list []string, init string, f func(string, string) string) string {
    acc := init
    for _, v := range list {
        acc = f(acc, v)
    }
    return acc
}

// avg 带错误约定的统计函数:空列表返回错误
func avg(list []float64) (float64, error) {
    if len(list) == 0 {
        return 0, errors.New("空列表无法求平均")
    }
    return reduceFloat(list, 0, func(a, b float64) float64 { return a + b }) / float64(len(list)), nil
}

```

```

func main() {
    temps := []float64{36.5, 37.1, 38.4, 39.0, 36.9, 37.8, 40.1}

    // 闭包:阈值可配置的过滤
    threshold := 37.0
    fever := filterFloat(temps, func(t float64) bool { return t > threshold })

    // 闭包:统计调用次数的映射(捕获的计数变量跨调用保留)
    count := 0
    rounded := mapFloat(temps, func(t float64) float64 {
        count++
        return math.Round(t*10) / 10
    })

    // reduce 组合:求和、求平均
    sum := reduceFloat(temps, 0.0, func(a, b float64) float64 { return a + b })
    a, err := avg(temps)
    if err != nil {
        fmt.Println("平均失败:", err)
        return
    }

    // 字符串管线
    names := []string{"张三", "李四", "王五"}
    joined := reduceStr(names, "", func(a, b string) string {
        if a == "" {
            return b
        }
        return a + "、" + b
    })

    // 匿名函数立即调用:求最大值
    max := func() float64 {
        m := temps[0]
        for _, t := range temps {
            if t > m {
                m = t
            }
        }
        return m
    }()

    fmt.Printf("发热(体温>%.1f):共 %d 人,%v\n", threshold, len(fever), fever)
    fmt.Printf("映射调用次数: %d,四舍五入后: %v\n", count, rounded)
    fmt.Printf("平均 %.2f,最高 %.1f,总和 %.2f\n", a, max, sum)
    fmt.Println("拼接结果:", joined)

    // 空列表错误约定演示
    if _, err := avg([]float64{}); err != nil {
        fmt.Println("空列表演示:", err)
    }

    // 链式组合:过滤 + 映射 + 归约
    result := reduceFloat(

```



```
mapFloat(  
    filterFloat(temps, func(t float64) bool { return t > 39 })),  
    func(t float64) float64 { return t - 37 },  
)  
0.0,  
func(a, b float64) float64 { return a + b },  
)  
fmt.Printf("高烧(高于 39 度)超出正常体温总和: %.1f 度\n", result)  
}
```

设计说明:三个高阶函数保持纯函数风格(不修改入参、返回新切片),组合出任意管线;闭包承担"可配置 + 有状态"两个职责——阈值在过滤条件里被捕获,计数器在映射函数里被捕获;错误约定用 (`float64, error`) 表达空输入,与 `errors.New` 配套。若把 `filterFloat` 等改成泛型(第 10 章),一套代码可服务 `int/string` 等任意类型——实验完成后可以尝试重写为泛型版本,体会"类型参数消灭重复代码"。

下一章预告:第5章 内存管理。函数学会了,接下来看内存:Go 的 GC 三色标记与并发回收、逃逸分析(变量可能偷偷跑到堆上!)、GOGC 调优、pprof 内存画像,以及 GC 语言里内存泄漏的真实形态——goroutine 泄漏。